

Data Engineering003 — Spark Assignment

Student	Suyash Maheshwari
Program	B.Tech Information Technology — JECRC Foundation
Tool	PySpark 4.1.2 Python 3 Local[*] Mode
Output	PySpark Code + Execution Results + Performance Insights

1. Spark Architecture & Execution Model

Component	Role	Key Detail
Driver	Hosts SparkContext; schedules tasks; collects results	One per application
Cluster Manager	Allocates cluster resources	YARN / Mesos / K8s / Standalone
Executors	JVM workers; execute tasks; cache RDD/DataFrame data	Multiple per node
SparkContext	Entry point to Spark functionality	Deprecated in Spark 3 → SparkSession
SparkSession	Unified entry point (SQL + Streaming + ML)	Use .builder pattern

Execution Modes

Mode	Driver Location	Use Case
Local[*]	Local JVM (all cores)	Development / Unit Testing
Client	Submitting Machine	Interactive: Notebooks, REPL
Cluster	Inside cluster node	Production workloads

Lazy Evaluation & DAG

Spark uses **Lazy Evaluation**: transformations are not executed immediately; instead, Spark records them in a **DAG (Directed Acyclic Graph)**. The DAG is only materialized when an **Action** is called (e.g., `show()`, `count()`, `write()`). The **Catalyst Optimizer** analyzes the DAG and applies optimizations like **Predicate Pushdown** (filters applied at source) and **Column Pruning** before execution.

■ Transformation (lazy): `filter`, `select`, `withColumn` — Action (eager): `show`, `count`, `write`, `collect`

2. SparkSession Initialization

```
spark = SparkSession.builder \ .appName("DataEngineering003_SparkAssignment") \
.master("local[*]") \ .config("spark.sql.shuffle.partitions", "4") \ .getOrCreate()
spark.sparkContext.setLogLevel("ERROR") print("SparkSession created | Version:",
spark.version) # >> SparkSession created | Version: 4.1.2
```

■ *shuffle.partitions=4 is set for local mode; default is 200, which adds overhead on small datasets.*

3. Schema Definition & Sample Dataset

```
schema = StructType([ StructField("emp_id", IntegerType(), True), StructField("emp_name",
StringType(), True), StructField("department", StringType(), True), StructField("salary",
FloatType(), True), StructField("join_date", StringType(), True), StructField("city",
StringType(), True), StructField("experience_yr", IntegerType(), True), ]) df =
spark.createDataFrame(data, schema=schema)
```

emp_id	emp_name	department	salary	join_date	city	exp_yr
101	Suyash Maheshwari	Engineering	85000	2021-06-15	Jaipur	3
102	Priya Mehra	Data Science	92000	2020-03-10	Bangalore	5
103	Rahul Verma	Engineering	78000	2022-01-20	Delhi	2
104	Ananya Singh	HR	55000	2019-07-01	Mumbai	6
105	Karan Joshi	Data Science	110000	2018-11-05	Hyderabad	7
106	Divya Patel	Engineering	88000	2021-09-30	Pune	4
107	Arjun Nair	Finance	67000	2020-12-15	Chennai	4
108	NULL	Engineering	72000	2023-03-01	Jaipur	1
109	Meera Iyer	Data Science	NULL	2022-08-18	Bangalore	2
110	Vikram Chauhan	HR	51000	2017-04-22	Delhi	8

■ *Intentional nulls introduced: emp_name (row 108) and salary (row 109) for null-handling demonstration.*

4. Null Value Detection & Handling

```
# Detect nulls per column null_counts = df.select([ F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in df.columns ]) null_counts.show() # Drop rows where emp_name is null; fill missing salary with 80000 df_clean = df.dropna(subset=["emp_name"]) \ .fillna({"salary": 80000.0})
```

emp_id	emp_name	department	salary	join_date	city	experience_yr
0	1	0	1	0	0	0

■ `dropna(subset=[...])` removes rows with nulls in specific columns only. `fillna()` imputes numeric columns.

5. Filtering, Selection & Column Renaming

```
# Filter: Engineering department with salary > 80000 df_filtered = df_clean \ .filter((F.col("department") == "Engineering") & (F.col("salary") > 80000)) \ .select("emp_id", "emp_name", "department", "salary", "city") # Rename columns df_renamed = df_clean \ .withColumnRenamed("emp_name", "employee_name") \ .withColumnRenamed("experience_yr", "years_of_experience")
```

emp_id	emp_name	department	salary	city
101	Suyash Maheshwari	Engineering	85000.0	Jaipur
106	Divya Patel	Engineering	88000.0	Pune

■ `select()` is a narrow transformation — no shuffle. `filter()` with predicate pushdown can be applied at read time for Parquet files.

6. DataFrame Transformations — Adding Computed Columns

```
df_transformed = df_clean \ .withColumn("salary_grade", F.when(F.col("salary") >= 90000, "A") \
    .when(F.col("salary") >= 70000, "B") .otherwise("C")) \ .withColumn("annual_bonus",
    F.round(F.col("salary") * 0.10, 2)) \ .withColumn("total_comp", F.round(F.col("salary") +
    F.col("salary") * 0.10, 2)) \ .withColumn("join_year", F.year(F.to_date(F.col("join_date"),
    "yyyy-MM-dd")))
```

emp_name	salary	grade	bonus	total_comp	join_yr
Suyash Maheshwari	85000	B	8500	93500	2021
Priya Mehra	92000	A	9200	101200	2020
Rahul Verma	78000	B	7800	85800	2022
Ananya Singh	55000	C	5500	60500	2019
Karan Joshi	110000	A	11000	121000	2018
Divya Patel	88000	B	8800	96800	2021
Arjun Nair	67000	C	6700	73700	2020
Meera Iyer	80000	B	8000	88000	2022
Vikram Chauhan	51000	C	5100	56100	2017

■ `withColumn()` is a narrow transformation — operates row-by-row with no shuffle required.

7. Aggregations — Wide Transformation (Shuffle)

```
dept_stats = df_transformed.groupBy("department") \ .agg(
    F.count("emp_id").alias("headcount"), F.round(F.avg("salary"), 2).alias("avg_salary"),
    F.max("salary").alias("max_salary"), F.min("salary").alias("min_salary"),
    F.round(F.sum("total_comp"), 2).alias("total_compensation") ) \ .orderBy("avg_salary",
    ascending=False)
```

Department	Count	Avg Salary	Max Salary	Min Salary	Total Comp
Data Science	3	94000.0	110000.0	80000.0	310200.0
Engineering	3	83666.67	88000.0	78000.0	276100.0
Finance	1	67000.0	67000.0	67000.0	73700.0
HR	2	53000.0	55000.0	51000.0	116600.0

■ `groupBy()` triggers a **SHUFFLE** — all rows for a key must be co-located. Controlled by `spark.sql.shuffle.partitions` (set to 4 in this job to avoid 200-partition overhead on small data).

8. Window Functions — Intra-Partition Ranking

```
from pyspark.sql.window import Window window_spec =  
Window.partitionBy("department").orderBy(F.desc("salary")) df_ranked = df_transformed \  
.withColumn("dept_salary_rank", F.rank().over(window_spec))
```

emp_name	department	salary	rank
Karan Joshi	Data Science	110000.0	1
Priya Mehra	Data Science	92000.0	2
Meera Iyer	Data Science	80000.0	3
Divya Patel	Engineering	88000.0	1
Suyash Maheshwari	Engineering	85000.0	2
Rahul Verma	Engineering	78000.0	3
Arjun Nair	Finance	67000.0	1
Ananya Singh	HR	55000.0	1
Vikram Chauhan	HR	51000.0	2

■ Window functions like `rank()`, `row_number()`, `lag()`, `lead()` operate within partitions — they DO trigger a shuffle over the partition key.

9. File Format Comparison — CSV vs Parquet

```
# Write df_transformed.coalesce(1).write.mode("overwrite") \ .option("header",  
True).csv("/output/employees_csv") df_transformed.coalesce(1).write.mode("overwrite") \  
.parquet("/output/employees_parquet") # Read back – schema preserved in Parquet df_back =  
spark.read.parquet("/output/employees_parquet") df_back.printSchema()
```

Feature	CSV	Parquet
Format	Row-based, plain text	Columnar, binary
Schema	Inferred (error-prone)	Embedded in file
Compression	None / manual	Snappy (default)
Read Performance	Full row scan	Column pruning + predicate pushdown
Write Performance	Fast	Slightly slower (encoding)
Best for	Export / human-readable	Analytics / large datasets
Null handling	Manual	Native

■ Parquet is the industry default for big data — columnar storage means Spark only reads the columns your query needs, not entire rows.

10. Performance Concepts & Best Practices

Concept	Description	Impact
Lazy Evaluation	Transformations deferred until action triggers execution	Allows Catalyst to optimize the full plan
DAG Lineage	Tracks all transformations for fault tolerance + replay	RDD recomputed from source on failure
Predicate Pushdown	Filters pushed to data source (Parquet/JDBC) level	Reads far less data from disk
Shuffle	Data redistribution across partitions (wide transforms)	Most expensive operation — minimize joins/groupBy
Partition Tuning	shuffle.partitions default=200; reduce for small data	Fewer partitions = less overhead on small jobs
coalesce() vs repartition()	coalesce() reduces partitions without full shuffle	Prefer coalesce() for writing outputs
show() not collect()	collect() pulls ALL data to driver — OOM risk	Always use show(n) for exploration
Column Pruning	Catalyst removes unneeded columns automatically	Reduces memory and I/O

11. End-to-End Pipeline Summary

Step	Operation	Type	Output
1	createDataFrame()	Action	Raw DataFrame (10 rows, 7 cols)
2	dropna() + fillna()	Transform	Clean DataFrame (9 rows)
3	filter() + select()	Transform	Engineering > 80k (2 rows)
4	withColumnRenamed()	Transform	Renamed columns
5	withColumn() x4	Transform	+salary_grade, bonus, total_comp, join_year
6	groupBy().agg()	Wide/Shuffle	Dept stats (4 rows)
7	Window rank()	Wide	Salary rank per dept
8	write.csv()	Action	CSV files on disk
9	write.parquet()	Action	Parquet files on disk
10	read.parquet()	Action	Schema verified — all 11 columns intact

All code executed successfully on PySpark 4.1.2 | Local[*] Mode | Java 17